

Replication Report: (title not recorded)

OSTI 2587225 | Coverage 7/10, Agreement 8/10

Rick Stevens

Ollie (AI Assistant)
Argonne National Laboratory

2026-04-27

1 Source paper

- **Title:** (title not recorded)
- **Authors:** Not recorded
- **Year:** n/a
- **Domain:** Not recorded
- **Identifier:** OSTI 2587225

2 Replication objective

The central claim of the paper concerns not recorded. Our objective was to reproduce the headline numerical/qualitative results within the constraints of the AI-driven Replication Project (24–72h, commodity GPU/CPU compute, no author contact). A replication plan is archived at 2587225-ScaWL-Scaling-k-WL-Weisfeiler-Lehman-Algorithms-in/replication.plan.pdf.

3 Methodology

We followed the standard Replication Project workflow: ingest the source PDF, extract method and data pointers, draft a replication plan, attempt the smallest end-to-end pipeline that produces a comparable number, then scale up where compute and data permit. Tools used in this replication are catalogued in the meta-paper’s computational tools inventory (see `COMPUTATIONAL_TOOLS_INVENTORY.md`).

This paper went through the Tier-Lift pipeline; supplementary notes at `.openclaw/workspace/24h-progress/tie`

Paper-directory README excerpt

ScaWL Replication (OSTI 2587225)

****Paper:**** C. Soss et al., ***ScaWL:** Scaling k-WL (Weisfeiler–Lehman) Algorithms in Distributed-Memory.* OSTI 2587225.

Status: Complete — **Score 9/10** (lifted 2026-04-28: added distributed C++/MPI 3-WL run on JLSE chiatta00)

Independent Python/NumPy re-implementation of 2-WL color refinement with ‘multiprocessing’-based parallelism; strong-scaling benchmark reproduces the qualitative shape of the paper’s Table 4 on a 10-core workstation.

— p —	time (s)	— speedup (ours) —	— speedup (paper avg) —	— — — — —	:	— — — — —	:	— — — — —
—:	— 1 —	5.82 —	1.00 —	1.00 —	— 2 —	3.08 —	1.89 —	2.38 —
0.94 —	6.17 —	7.64 —	— 16 —	0.79 —	7.41 —	13.20*	—	—

Correctness (all pass)

- Iso pair (random 4-regular, $n=30$ with relabeling)
- Non-iso 2-regular pair C12 vs $2 \times C6$ (distinguished by 2-WL but not 1-WL)
- Strongly-regular pair (Shrikhande vs 4×4 rook) — match expected (2-WL cannot distinguish)

[code block omitted]

[code block omitted]

The replication was driven by an OpenClaw agent loop (Claude Opus / GPT-5 class models) issuing shell, Python, and (where applicable) GPU jobs. Compute usage and wall-time for individual papers are summarised in the meta-paper’s resource table; not separately recorded for this paper unless noted in the Tier-Lift artefact. Sub-agents were spawned for replication-plan generation and (for papers in batches 1–4) for tier-lift extension runs.

- Not recorded.

- Not recorded.

The dominant blocker categories for this paper, mapped onto the meta-paper’s 9-category friction taxonomy (§4), are listed in §?? below.

Per-quantity numerical comparisons are not separately tabulated in the structured evaluation record for this paper beyond the agreement rationale below; where the rationale references specific numbers, they are reproduced verbatim. A full numerical comparison table is recorded in the per-replication artefacts (see §??). For papers below 8/8, the agreement rationale identifies the specific quantities that diverge.

2

7 Score and friction-point tagging

- **Coverage:** 8/10 — 2-WL re-implementation + correctness oracles + strong-scaling reproduction (single node); plus distributed 3-WL C++/MPI run on chiatta00 with strong-scaling sweep at $n \in \{30, 60, 80, 100\}$, $P \in \{1, 2, 4, 8, 16, 32, 64\}$. Multi-node InfiniBand runs and full UFL benchmark suite still out of scope.
- **Agreement:** 9/10 — Qualitative shape of the strong-scaling curve reproduced for both 2-WL and 3-WL on the available single-node hardware. End-to-end $26.4\times$ speedup at $P=8 \times \text{OMP}=16$ on chiatta00 ($n=100$). Paper’s headline $2,193\times$ multi-node figure not attempted (out of scope on one node). Memory feasibility of 3-WL confirmed: ≈ 8 MB at $n=100$, refuting the prior report’s ~ 100 GB Python-overhead estimate by four orders of magnitude.

Friction points encountered (taxonomy tags):

- **F5:** Force-field / hyperparameter drift (unspecified configuration)

8 Follow-on questions

- Not recorded.

9 Reproducibility pointers

- **Paper directory:** 2587225-ScaWL-Scaling-k-WL-Weisfeiler-Lehman-Algorithms-in
- **Replication artefacts:**
 - /Users/stevens/Dropbox/REPLICATE-PROJECT/2587225-ScaWL-Scaling-k-WL-Weisfeiler-Lehman-
- **Replication-dir hint from JSONL:** /Users/stevens/Dropbox/REPLICATE-PROJECT/2587225-ScaWL-Sca
- **Status:** tier_lift_v2_2026_04.27

To re-run, change directory into the paper directory above and consult its README.md for the entry-point script. Most replications follow the convention `replication/run_*.sh` or `replication/<subdir>/run.py`.